WebGL avanzado (I)

Área de Arquitectura y Tecnología de Computadores – Versión 1.2.12455, 09/02/2026

Índice

1. Introducción
2. Texturas avanzadas
3. Texturas de vídeo
4. Controlando variables de la escena
5. Controles para determinar el *framerate*
6. Movimiento de la cámara
Entregar

1. Introducción

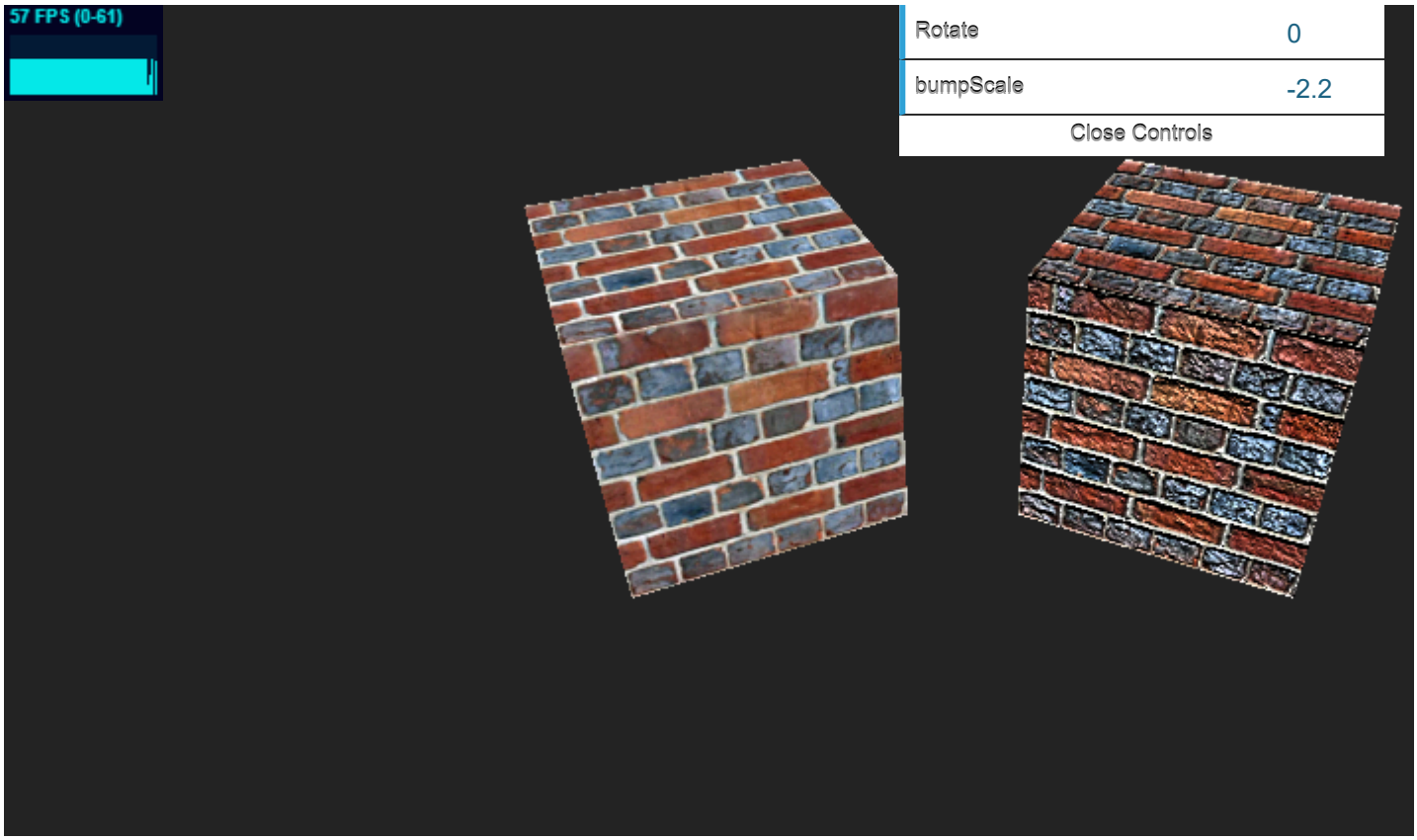
En las sesiones anteriores trabajaste con escenas 3D básicas y texturas que permitían hacer más realistas los diferentes objetos. Además, aprendiste a animar las escenas para ir un paso más allá de las imágenes estáticas y cargar modelos 3D generados con programas de modelado.

En esta sesión ahondaremos en la funcionalidad de WebGL y características adicionales de three.js. Concretamente, introduciremos los mapas topológicos para producir un efecto de profundidad en las texturas; utilizaremos texturas de vídeo; añadiremos controles para modificar parámetros de la escena; y modificaremos la posición de la cámara para simular el desplazamiento dentro de la escena.

2. Texturas avanzadas

Como hemos visto en la sesión anterior, se pueden añadir texturas para darle más realismo a los objetos. Si bien añadir un mapa de textura básico en ocasiones basta para conseguir un efecto realista, otras hay que ir un poco más allá. Vamos a verlo con un ejemplo. La siguiente escena requiere el uso de un mapa topológico para hacer más realista el cubo.

Efecto de un mapa de alturas



- Utiliza los controles de la escena para comprobar el efecto sobre la textura del parámetro `bumpScale` en el cubo de la derecha y compárala con la textura simple del cubo de la izquierda. ¿Puedes apreciar el efecto sobre la textura? Ten en cuenta que en la escena anterior se muestran dos objetos con la misma geometría (dos cubos) y el uso de este tipo de texturas permite simular cambios en la superficie (rugosidad) de uno de los cubos.

Existen varias formas para darle cierta sensación de volumen a las texturas de forma que el efecto visual sea más realista. Una de ellas es mediante el uso de mapas topológicos. En ese caso, se utilizarán dos texturas.

La siguiente textura es la básica de ambos cubos.

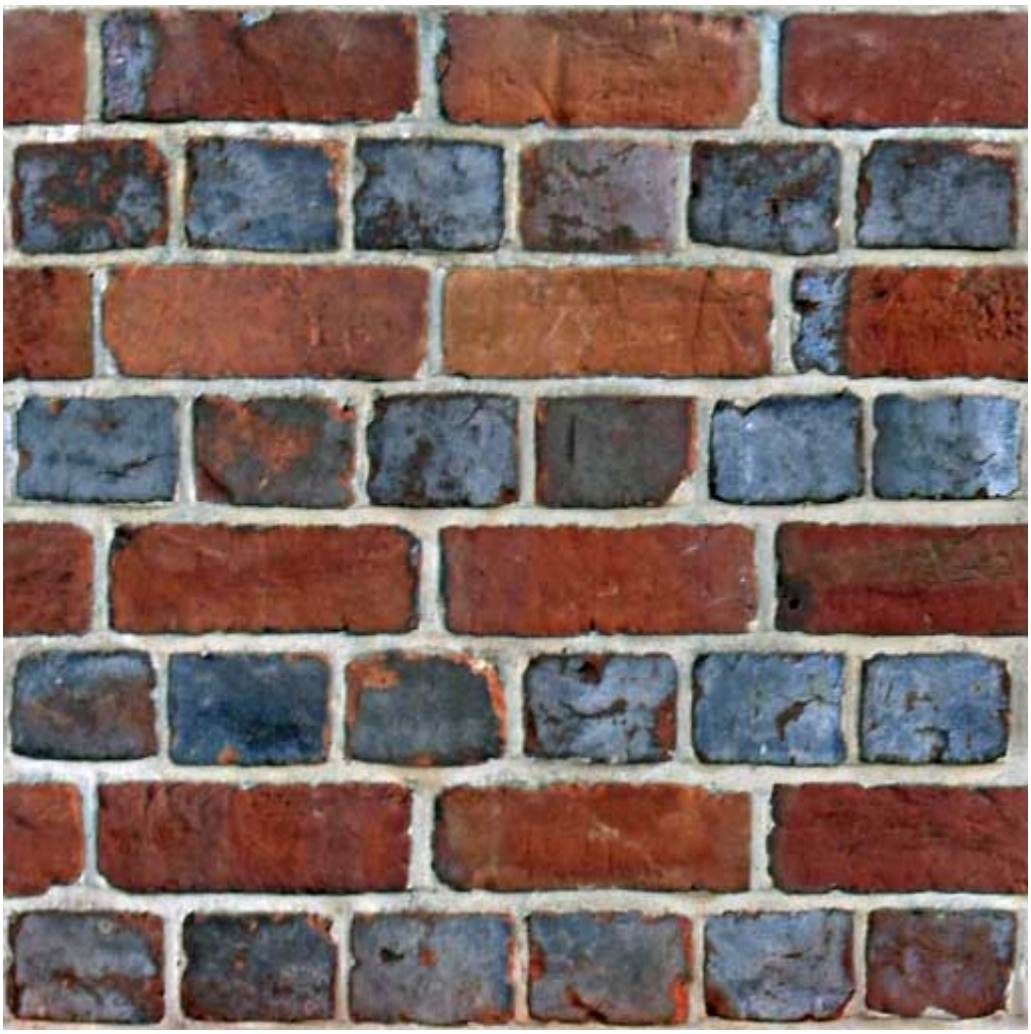


Figura 1. Textura básica del cubo

El mapa topológico es una textura en escala de grises donde un color cerca del blanco indica mayor altura y un color cercano al negro indica menor altura. El mapa topológico utilizado es el siguiente.

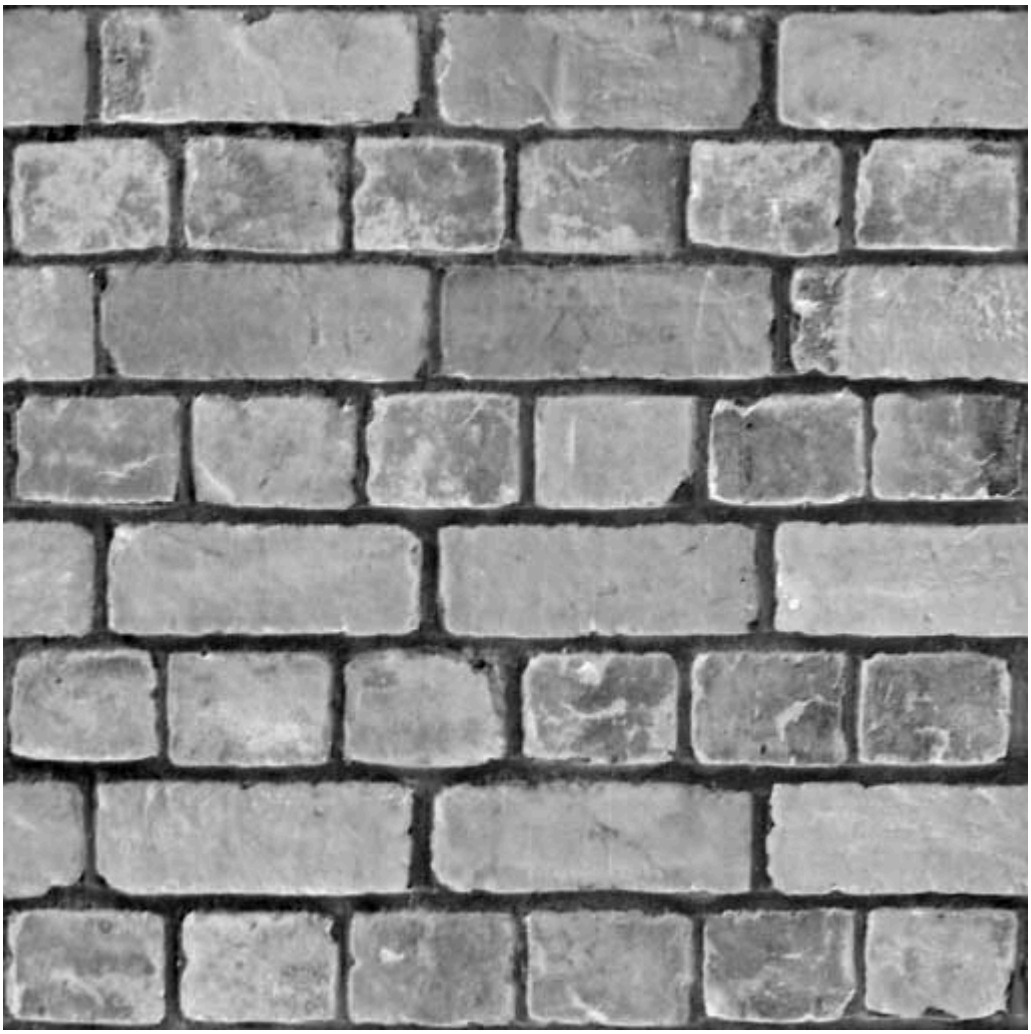


Figura 2. Mapa topológico del cubo

Es posible especificar varias texturas al definir un material con el siguiente código:

JAVASCRIPT

```
const textureLoader = new THREE.TextureLoader( );  // The object used to load textures
const material = new THREE.MeshPhongMaterial(
    {
        map: textureLoader.load( "brick.jpg" ),
        bumpMap: textureLoader.load( "brick-map.jpg" )
    } );
```

- Crea un nuevo proyecto para esta sesión práctica.
- Crea una copia de la página web `prac1-3.html` desarrollada en la primera sesión y guárdala como `prac3-1.html`.

> Es posible que recibas un aviso al transpilar en relación al gran tamaño del *bundle* generado. Puedes eliminar este aviso añadiendo el siguiente código a la configuración de Webpack en el fichero `webpack.config.js`.
>
> JAVASCRIPT
>
> ```
> performance: {
>  hints: false,
>  maxAssetSize: 1000000,
>  maxEntrypointSize: 1000000
> }
> ```

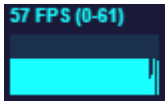
- Anima la escena para hacer girar el cubo sobre el eje y.
- Modifica el cubo para añadirle la textura y el mapa topológico indicados anteriormente.

3. Texturas de vídeo

Una característica interesante de WebGL, y de three.js por extensión, es la posibilidad de utilizar texturas de vídeo. Esto nos permite utilizar un vídeo (o incluso una cámara web) como textura en un objeto 3D.

Vamos a realizar una escena parecida a la siguiente.

Textura de vídeo



Open Controls

El primer paso consiste en la definición de la fuente de vídeo. En este caso, basta con definir un elemento html `video` de la siguiente forma.

HTML
```
<video id="video" class="videot" style="display:none" autoplay="true">
    <source src="sintel_trailer-480p.mp4">
    <source src="sintel_trailer-480p.ogv">
</video>
```

El marcado anterior utiliza a modo de ejemplo el trailer de la película Sintel (https://download.blender.org/durian/trailer/) de la *Blender Foundation*. Opcionalmente, puedes descargar ambos vídeos utilizando estos enlaces: formato mp4 (https://www.atc.uniovi.es/master/2cgsm/sintel.mp4) y formato ogv (https://www.atc.uniovi.es/master/2cgsm/sintel.ogv).

Se utilizan dos formatos diferentes para soportar diferentes navegadores. Abordaremos esta cuestión en posteriores prácticas.

Fíjate que el elemento `video` se define oculto para el navegador procese el vídeo y lo reproduzca sin mostrarlo en pantalla. La idea es renderizar este vídeo en un canvas intermedio que posteriormente podrá ser utilizado como textura. Este canvas, cuyo tamaño debería ser el mismo que el del vídeo, puede definirse con el siguiente código.

JAVASCRIPT
```
const video = document.getElementById( 'video' );

const image = document.createElement( 'canvas' );
image.width = 480;  // Video width
image.height = 204; // Video height
const imageContext = image.getContext( '2d' );
imageContext.fillStyle = '#000000';
imageContext.fillRect( 0, 0, image.width - 1, image.height - 1 );
const texture = new THREE.Texture( image );
```

El ancho y alto asignados al objeto canvas creado deben coincidir con las dimensiones del vídeo. Podrían obtenerse automáticamente, pero eso requeriría atender el evento `loadedmetadata` del vídeo.

Tras esto, es necesario crear el plano que será texturizado. Para ello se puede utilizar un material básico sobre el que aplicar la textura. De esta forma no será necesario incluir luces en la escena.

JAVASCRIPT
```
const material = new THREE.MeshBasicMaterial( { map: texture } );
const wall = new THREE.Mesh( new THREE.PlaneGeometry( image.width, image.height, 4, 4 ), material );
```

Fíjate que el plano tiene las mismas dimensiones que el vídeo. En cualquier caso, si el ancho o alto no es potencia de 2 recibirás una advertencia en el navegador. Es más eficiente trabajar con texturas con tamaños de potencias de 2.

Ya por último, faltaría actualizar la textura en la función de animación de la escena, invocada a través de `requestAnimationFrame()`. Esto se hace en dos pasos: copiar el cuadro de vídeo actual al canvas y actualizar la textura.

JAVASCRIPT

```javascript
if ( video.readyState === video.HAVE_ENOUGH_DATA ) {

    imageContext.drawImage( video, 0, 0 );
    if ( texture ) texture.needsUpdate = true;
}
```

Fíjate que las variables utilizadas deben ser accesibles desde la función de animación de la escena.

Los navegadores modernos impiden la reproducción de audio y vídeo de forma automática antes de que el usuario interactúe con la página. Por este motivo es necesario definir un mecanismo que fuerce una interacción del usuario previa a la reproducción del vídeo. Lo más sencillo es incluir un botón que al ser pulsado ocasione la construcción de la escena completa.

- Crea una nueva escena de nombre `prac3-2.html` . Añade en ella un plano que use como textura un vídeo.
- Anima el plano para que rote y así apreciar el efecto de la textura.
- Añade una capa *overlay* para que la escena se renderice únicamente tras la pulsación de un botón. Puedes utilizar este ejemplo.

4. Controlando variables de la escena

En ocasiones interesa poder modificar interactivamente algún parámetro de la escena. Para ello puede utilizarse la biblioteca data.gui.js (https://github.com/dataarts/dat.gui), que permite crear controles para modificar dinámicamente parámetros de la escena. Esta biblioteca es la utilizada en las escenas mostradas en las anteriores secciones.

- Crea una copia de la escena `prac3-1.html` y llámala `prac3-3.html` . En esta escena añadiremos un control.
- Threejs ya incluye la biblioteca lil-gui. Impórtala utilizando el siguiente código.

JAVASCRIPT

```javascript
import { GUI } from 'three/examples/jsm/libs/lil-gui.module.min';
```

A modo de ejemplo, vamos a definir un control para modificar el parámetro `bumpScale` del material. Este parámetro, como has visto anteriormente, modifica el efecto del mapa topológico: es un factor de escala.

Para incluir los controles es necesario definir un objeto contenedor de las variables sobre las que operan los controles. En el caso de utilizar un único control para modificar el parámetro `bumpScale` puede usarse un código como el siguiente.

JAVASCRIPT

```javascript
const controlData = {
    bumpScale: material.bumpScale
}
```

Este código asume que la variable <code>material</code> referencia el material texturizado con el mapa topológico.

El atributo `bumpScale` de `controlData` podría llamarse de cualquier forma. Se llama igual que el parámetro a modificar por coherencia.

A partir de la definición anterior, para crear el control, bastaría con usar el siguiente código.

JAVASCRIPT

```
const gui = new GUI( );
gui.add( controlData, 'bumpScale', -4, 4 ).step(0.1).name( 'bumpScale' );
```

Fíjate en el código e intenta adivinar qué hace. La funcion `add()` recibe cuatro parámetros: el objeto contenedor, el nombre del atributo a modificar, y unos valores mínimo y máximo. Se pueden llamar a más funciones de `gui` de forma encadenada. En este caso se llaman a las funciones `step()`, para fijar el paso, y `name()`, para indicar el nombre a mostrar.

- Añade el control a la escena tal como se ha indicado.
- Prueba la escena, comprobarás que aparece el control y se modifica su valor pero no tiene ningún efecto sobre la escena.

Faltaría enlazar el valor de `controlData.bumpScale` con el valor de `bumpScale` del material. Esto puede hacerse dentro de la función de animación de la escena con el siguiente código.

JAVASCRIPT

```
material.bumpScale = controlData.bumpScale;
```

Fíjate que el objeto `controlData` debe ser accesible desde la función de animación de la escena. Deberás definirlo como una variable global.

- Añade el código para enlazar el control con la propiedad `bumpScale` del material.
- Prueba la escena. Ahora sí deberías observar el comportamiento esperado.

5. Controles para determinar el *framerate*

Otra biblioteca interesante es stats.js (https://github.com/mrdoob/stats.js/), que permite mostrar el número de cuadros por segundo con los que se renderiza la escena. Vamos a añadir este control a la escena `prac3-3.html` anteriormente creada.

- Al igual que la anterior, threejs ya incluye la biblioteca stats.js. Impórtala en tu escena `prac3-3.html` utilizando el siguiente código.

JAVASCRIPT

```
import Stats from 'three/examples/jsm/libs/stats.module';
```

Para incluir el control mostrando el *framerate* en la escena basta con usar el siguiente código.

JAVASCRIPT

```
const stats = new Stats( );
stats.dom.style.position = 'absolute';
stats.dom.style.top = '0px';
document.body.appendChild( stats.dom );
```

Se asume que el canvas del renderizador ha sido añadido al elemento `body` de la página.

- Añade un control de este tipo a la escena.
- Prueba la escena. Algo no va bien.

Faltaría actualizar el control dentro de la función de animación de la escena.

JAVASCRIPT

```
stats.update( );
```

Fíjate que el objeto `stats` debe ser accesible desde la función de animación de la escena. Deberás definirlo como una variable global.

- Modifica la escena para mostrar correctamente el *framerate*.

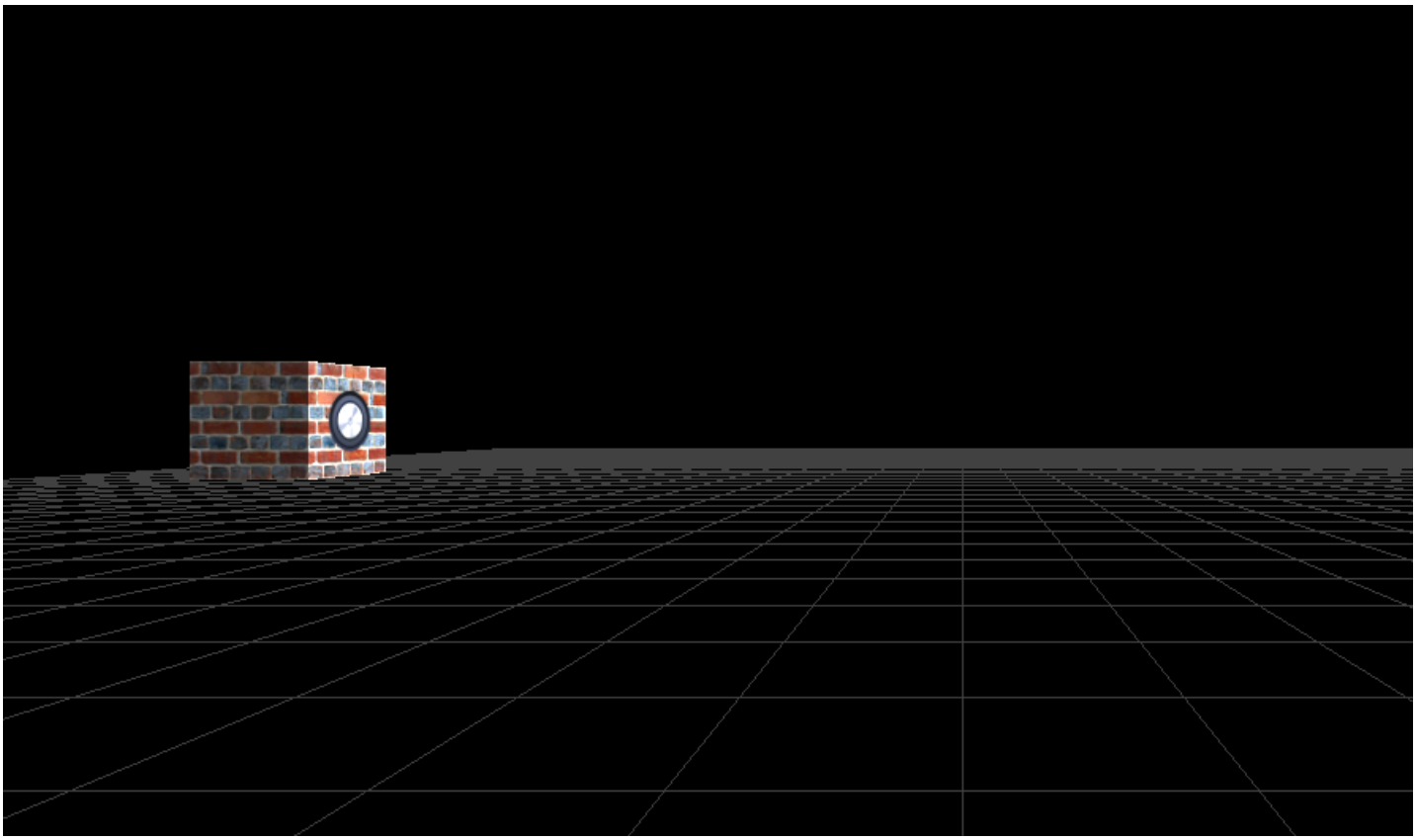
6. Movimiento de la cámara

La interactividad de las escenas 3D es un factor crítico a la hora de ofrecer una experiencia de usuario satisfactoria. La forma básica de interactuar con la escena que tiene el usuario es a través del teclado y del ratón.

El biblioteca three.js proporciona la API FirstPersonControls.js (https://github.com/mrdoob/three.js/blob/master/examples/js/controls/FirstPersonControls.js) para trabajar con la cámara fácilmente en aquellas escenas en primera persona (*first person*), donde la cámara se va desplazando por la escena de la misma forma que haría el observador.

Vamos a desarrollar la siguiente escena, que emula un escenario en primera persona, donde aparece una rejilla a modo de suelo y dos cubos texturizados. Esta escena será la base de algunas escenas que se desarrollarán en la siguiente sesión.

Escena de tipo primera persona



Puedes utilizar las flechas del teclado para moverte (o la combinación W-S-A-D) y el ratón para cambiar la vista. El botón izquierdo y derecho del ratón se pueden utilizar como alternativa al teclado para avanzar y retroceder respectivamente.

La escena está integrada en este documento utilizando un elemento `iframe`, por lo que no podrás utilizar el teclado para desplazarte. Prueba directamente sobre la escena.

- Crea una nueva escena de nombre `prac3-4.html` .
- Añade una rejilla a modo de suelo utilizando un grid del tipo `GridHelper` de la siguiente forma.

JAVASCRIPT

```
const helper = new THREE.GridHelper( 800, 40, 0x444444, 0x444444 );
helper.position.y = 0.1;
```

  El primer parámetro del constructor de `GridHelper` indica el ancho y largo de la rejilla, mientras que el segundo indica el paso. Los otros dos parámetros indican los colores de la línea central y del resto de líneas del grid.
- Antes de añadir los cubos a la escena es necesario definir una fuente de luz para que sean visibles. En esta escena utilizaremos una luz direccional blanca ubicada en la posición (0, 0.5, 100). Añade la luz a la escena.
- Añade ahora los dos cubos texturizados con un tamaño de 50 unidades. Se utilizan las dos texturas que se muestran a continuación: una principal y un mapa topológico.

Figura 3. Textura básica a utilizar en el cubo

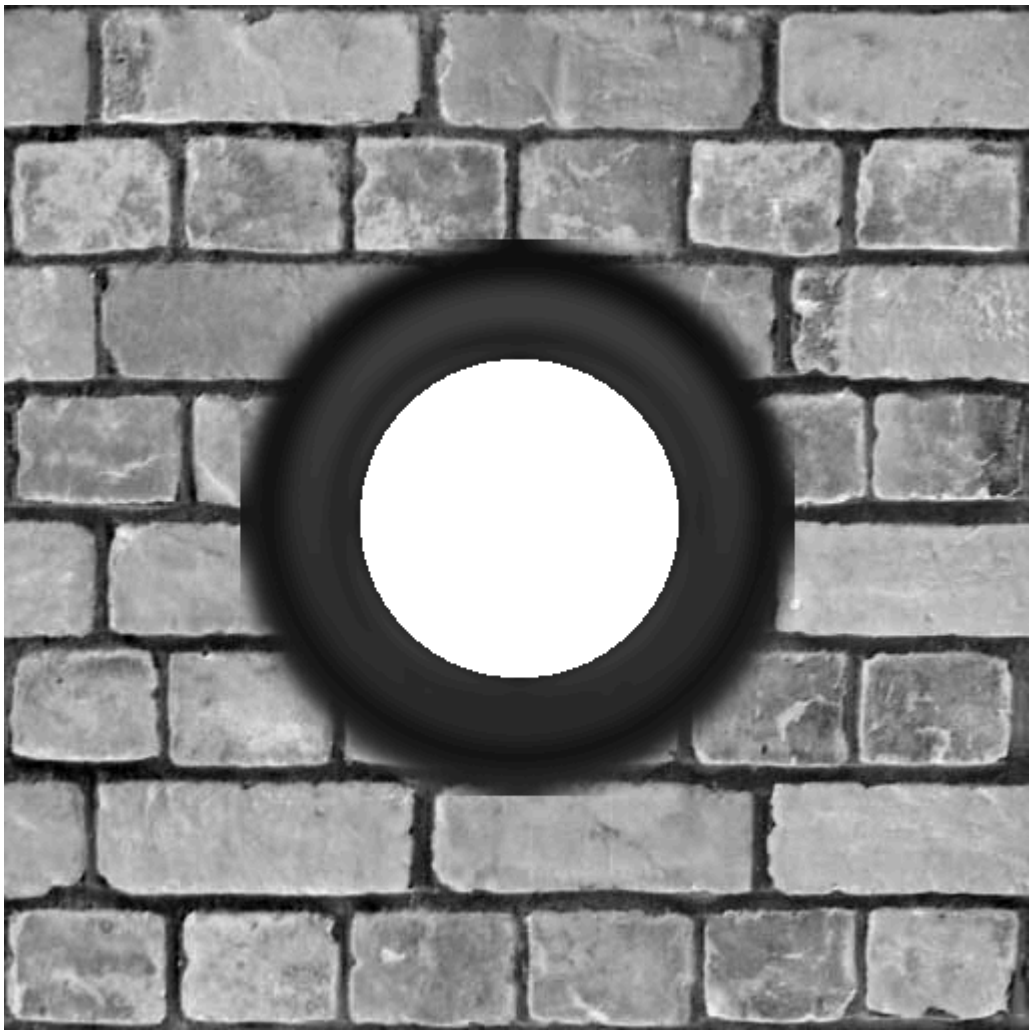


Figura 4. Mapa topológico a utilizar en el cubo

- Estos cubos tendrán una particularidad: una cara va a ser diferente del resto. Utilizaremos las texturas anteriores para una única cara de cada cubo y el resto de caras utilizarán las texturas de la sección Texturas avanzadas. Para ello, puede utilizarse el siguiente código.

JAVASCRIPT

```
const specialFaceMaterial = // Material for a face
const regularFaceMaterial = // Material for the rest of the faces

// A box has 6 faces
const materials = [
    specialFaceMaterial,
    regularFaceMaterial,
    regularFaceMaterial,
    regularFaceMaterial,
    regularFaceMaterial,
    regularFaceMaterial,
];

// Create the mesh with the material array as parameter
```

- Sitúa un cubo bien alejado del otro pero sin salirse del plano que simula el suelo de la escena y con las caras distintas enfrentadas.
- Visualiza la escena. Verás que la iluminación es muy pobre. Para solucionarlo, vamos a introducir una fuente de luz hemisférica, que simula la luz natural. Puedes hacerlo con el siguiente código.

```javascript
const hemiLight = new THREE.HemisphereLight( 0xffffff, 0xf0f0f0, 0.6 );
hemiLight.position.set( 0, 500, 0 );
scene.add( hemiLight );
```

- Por último, faltaría añadir interactividad a la escena introduciendo los controles de la cámara, para lo cual es necesario importar la biblioteca, que ya está incluida en threejs.

```javascript
import { FirstPersonControls } from 'three/examples/jsm/controls/FirstPersonControls.js';
```

- El uso de los controles de la cámara requiere dos pasos: definir los controles y actualizar los valores en el ciclo de renderizado de la escena. Añade los controles a la escena

```javascript
const controls = new FirstPersonControls( camera, renderer.domElement );
controls.movementSpeed = 70;
controls.lookSpeed = 0.05;
controls.noFly = false;
controls.lookVertical = false;
```

> Puedes jugar con los parámetros para ver su efecto.

- Para actualizar los controles necesitamos llevar registro del tiempo transcurrido en cada iteración del ciclo de renderizado. En primer lugar crearemos un reloj.

```javascript
const clock = new THREE.Clock( );
```

- Ahora ya dentro de la función de actualización dentro del ciclo de renderizado actualizamos los controles.

```javascript
const delta = clock.getDelta();
controls.update( delta );
```

- Prueba la escena. Deberías poder desplazarte por ella.

Entregar

Al finalizar la sesión se creará una página web (fichero html) dentro del proyecto a modo de memoria utilizando la plantilla de prácticas cubriendo los siguientes aspectos:

- Un resumen de las tareas realizadas en la sesión y los resultados obtenidos, detallando las dificultades surgidas durante su desarrollo y la forma de solucionarlas.
- TODAS las escenas generadas (cada una incluida dentro del objeto *canvas* correspondiente) y respuestas concisas correspondientes a las cuestiones planteadas en el guión.
- Las capturas de pantalla que consideres más significativas si hubiere lugar.

El repositorio git ya con la memoria incluida se exportará a un fichero ZIP y se subirá a la tarea correspondiente en la página web de la asignatura. El fichero zip deberá contener los siguientes ficheros:

- index.html (siguiendo la plantilla dada).

- report.css
- prac3-1.html a prac3-4.html
- style.css
- package.json
- package-lock.json
- webpack.config.js
- src/
  - prac3-1.js a prac3-4.js
- .vscode/
  - launch.js
- /videos
  - todos los vídeos utilizados
- textures/
  - todas las texturas utilizadas